LangChain ChatPromptTemplate — Complete Guide

PromptTemplate vs ChatPromptTemplate · Multiple input variables · chain.invoke() · Properly commented Python code

PromptTemplate | ChatPromptTemplate | 2+ Variables | chain.invoke() | Differences

1. What Are Prompt Templates?

In LangChain, a **prompt template** is a reusable blueprint for building messages to send to a language model. Instead of hardcoding a prompt string every time, you define a template once with placeholder variables, then fill them in at runtime. This keeps your code clean, reusable, and easy to test.

Think of it like Python's f-strings, but designed specifically for building AI prompts — with built-in validation, chaining support, and message role awareness.

The two main template classes

Class	What it builds	Best for
`PromptTemplate`	A single plain-text string	Completion models (older APIs), simple single-turn
`ChatPromptTemplate`	A list of role-based messages (system, human, AI)	Chat models like GPT-4, Claude, Gemini — the mo

2. PromptTemplate — The Basics

PromptTemplate produces a single flat string. You define a template with **{variable}** placeholders, and it fills them in when called. It was the original LangChain template class and is still useful for simple, single-turn use cases or non-chat models.

EXAMPLE 1 — PromptTemplate (single string)

```
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# EXAMPLE 1: PromptTemplate — single flat string
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

# pip install langchain langchain-openai

from langchain.prompts import PromptTemplate

# Step 1: Define the template
# Placeholders are written as {variable_name} inside the string.
# 'input_variables' lists every placeholder that must be filled.
template = PromptTemplate(
input_variables=["topic", "audience"], # two placeholders
template=(
"Explain {topic} in simple terms "
"suitable for a {audience}."
)
)

# Step 2: Format the template by providing values for every variable.
# .format() returns a plain Python string — not a message object.
formatted_prompt = template.format(
topic="recursion", # fills {topic}
audience="10-year-old" # fills {audience}
)

print(formatted_prompt)
# Output: Explain recursion in simple terms suitable for a 10-year-old.

# ■■ What PromptTemplate produces ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# It returns a plain string — no role (system/user/assistant).
# The model receives: "Explain recursion in simple terms suitable
# for a 10-year-old."
print(type(formatted_prompt)) # <class 'str'>
```

PromptTemplate.format() returns a plain Python string

Using PromptTemplate in a chain

EXAMPLE 2 — PromptTemplate in a chain with invoke()

```
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# EXAMPLE 2: PromptTemplate chained with a model using LCEL
# (LangChain Expression Language — the | pipe operator)
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

from langchain.prompts import PromptTemplate
from langchain_openai import ChatOpenAI # or any other LLM

# Define the prompt template with two input variables
prompt = PromptTemplate(
input_variables=["topic", "style"],
template="Write a {style} explanation of {topic}."
)

# Instantiate the language model
llm = ChatOpenAI(
model="gpt-4o-mini",
api_key="YOUR_OPENAI_API_KEY"
)

# Build the chain using the | (pipe) operator.
# Data flows left to right: prompt → llm
chain = prompt | llm

# Invoke the chain — pass a dict with values for every variable
response = chain.invoke({
"topic": "neural networks", # fills {topic}
"style": "beginner-friendly" # fills {style}
})

# .content holds the model's text reply
print(response.content)
```

3. ChatPromptTemplate — The Modern Standard

ChatPromptTemplate builds a *list of role-based messages* — system, human, and AI — instead of a single flat string. Modern chat models (Claude, GPT-4, Gemini) expect this structure. Each message has a **role** (who is speaking) and **content** (what they say, which can contain {variables}).

The three message roles

Role	Tuple syntax	What it represents
`system`	`("system", "...")`	Instructions that shape the AI's behaviour and persona
`human`	`("human", "...")`	The user's message or question
`ai`	`("ai", "...")`	A prior AI response (for multi-turn conversations)

Basic ChatPromptTemplate — 2 input variables

EXAMPLE 3 — ChatPromptTemplate with 2 variables

```
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# EXAMPLE 3: ChatPromptTemplate with 2 input variables
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

from langchain.prompts import ChatPromptTemplate

# Step 1: Create the template using a list of (role, content) tuples.
# Variables are written as {variable_name} — same syntax as PromptTemplate.
# Here we have TWO variables: {language} and {topic}
chat_prompt = ChatPromptTemplate.from_messages([
# The system message sets the AI's role/persona
("system", "You are an expert {language} programmer."),

# The human message is the actual question, with a second variable
("human", "Explain the concept of {topic} in {language}.")
])

# Step 2: Format the template — pass a dict with all variable values
formatted_messages = chat_prompt.format_messages(
language="Python", # fills {language} in BOTH messages
topic="decorators" # fills {topic} in the human message
)

# Step 3: Inspect what was produced — a list of message objects
for msg in formatted_messages:
print(f"[{msg.type.upper()}]: {msg.content}")

# Output:
```

```
# [SYSTEM]: You are an expert Python programmer.
# [HUMAN]: Explain the concept of decorators in Python.
```

format_messages() returns a list of typed message objects, not a plain string

OUTPUT

```
[SYSTEM]: You are an expert Python programmer.
[HUMAN]: Explain the concept of decorators in Python.
```

4. Using chain.invoke() with Multiple Variables

When you chain a **ChatPromptTemplate** to a model using the **|** (pipe) operator, you call **chain.invoke()** with a dictionary containing values for every variable in the template. This is the standard LangChain Expression Language (LCEL) pattern.

EXAMPLE 4 — chain.invoke() with 2 variables (core pattern)

```
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# EXAMPLE 4: chain.invoke() with 2 input variables
# This is the core pattern — learn this one first.
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

from langchain.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI

# ■■ Step 1: Define the ChatPromptTemplate ■■■■■■■■■■■■■■■■■■■■■■■■■
# Two variables: {language} and {topic}
# The system message gives the AI a specialised role.
# The human message asks the actual question.
prompt = ChatPromptTemplate.from_messages([
("system", "You are a helpful {language} programming tutor."),
("human", "Explain {topic} with a short code example.")
])

# ■■ Step 2: Create the model ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
llm = ChatOpenAI(
model="gpt-4o-mini",
temperature=0.3, # lower = more focused answers
api_key="YOUR_OPENAI_API_KEY"
)

# ■■ Step 3: Build the chain ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# The | operator connects the prompt to the model.
# Data flows: prompt → llm
chain = prompt | llm

# ■■ Step 4: Invoke the chain ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Pass a dict with keys matching EVERY {variable} in the template.
response = chain.invoke({
"language": "Python", # fills {language} in both messages
"topic": "list comprehensions" # fills {topic} in human message
})

# ■■ Step 5: Access the response ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# chain.invoke() returns an AIMessage object.
# .content holds the model's reply as a plain string.
```

```
print(response.content)
```

Using 3 input variables

EXAMPLE 5 — chain.invoke() with 3 input variables

```
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# EXAMPLE 5: ChatPromptTemplate with 3 input variables
# Adding a third variable {level} to control response depth
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
from langchain.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
# Three variables: {role}, {topic}, {level}
prompt = ChatPromptTemplate.from_messages([
# System message — defines the AI's expertise
("system",
"You are a {role}. Always tailor your explanations to a {level} audience."),
# Human message — the question with its own variable
("human",
"Can you explain {topic} clearly?")
])
llm = ChatOpenAI(model="gpt-4o-mini", api_key="YOUR_OPENAI_API_KEY")
# Build the LCEL chain
chain = prompt | llm
# Invoke with all 3 variables
response = chain.invoke({
"role": "senior data scientist", # fills {role}
"level": "beginner", # fills {level}
"topic": "gradient descent" # fills {topic}
})
print(response.content)
```

Using 4 input variables — full control

EXAMPLE 6 — chain.invoke() with 4 input variables

```
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# EXAMPLE 6: ChatPromptTemplate with 4 input variables
# Full control over persona, format, language, and task
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

```
from langchain.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI

# Four variables: {persona}, {language}, {task}, {format}
prompt = ChatPromptTemplate.from_messages([
("system",
"You are a {persona}. Respond in {language}. "
"Always format your output as {format}."),

("human",
"Please help me with the following: {task}")
])

llm = ChatOpenAI(model="gpt-4o-mini", api_key="YOUR_OPENAI_API_KEY")

chain = prompt | llm

# Invoke with all 4 variables — each key matches a {variable}
response = chain.invoke({
"persona": "friendly Python tutor", # fills {persona}
"language": "English", # fills {language}
"format": "numbered steps", # fills {format}
"task": "How do I read a CSV file?" # fills {task}
})

print(response.content)
```

5. Adding an Output Parser to the Chain

By default, **chain.invoke()** returns an **AIMessage** object. Adding a **StrOutputParser** to the end of the chain makes it return a plain Python string directly — which is usually what you want.

EXAMPLE 7 — Full chain with StrOutputParser

```
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# EXAMPLE 7: Full chain with StrOutputParser
# prompt | llm | output_parser → returns a plain string
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

from langchain.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

# ■■ Define the prompt with 3 variables ■■■■■■■■■■■■■■■■■■■■■■■■■■■
prompt = ChatPromptTemplate.from_messages([
# System: set the AI's tone and role
("system",
"You are a {tone} assistant who specialises in {domain}."),

# Human: the actual request
("human",
"Give me a {length} explanation of {concept}.")
])

# ■■ Model ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
llm = ChatOpenAI(model="gpt-4o-mini", api_key="YOUR_OPENAI_API_KEY")

# ■■ Output parser: converts AIMessage → plain string ■■■■■■■■■■■■■■
output_parser = StrOutputParser()

# ■■ Build the full chain: prompt → llm → parser ■■■■■■■■■■■■■■■■■■■
# The pipe operator chains them in order.
chain = prompt | llm | output_parser

# ■■ Invoke: the result is now a plain str, not an AIMessage ■■■■■■■
result = chain.invoke({
"tone": "friendly", # fills {tone}
"domain": "machine learning",# fills {domain}
"length": "one-paragraph", # fills {length}
"concept": "overfitting" # fills {concept}
})

# result is a str — no .content needed
print(type(result)) # <class 'str'>
print(result)
```

prompt | llm | StrOutputParser() → returns a plain Python string

6. PromptTemplate vs ChatPromptTemplate — Full Comparison

This is the most important distinction to understand in LangChain prompting. The two classes look similar but produce fundamentally different outputs and are designed for different model types.

- Produces a flat string

One message, no roles, suited for completion-style models

EXAMPLE 8a — PromptTemplate output (plain string)

```
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# EXAMPLE 8a: PromptTemplate — what it produces
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

from langchain.prompts import PromptTemplate

# Define a template with two variables
pt = PromptTemplate(
input_variables=["subject", "grade"],
template="Explain {subject} suitable for a grade {grade} student."
)

# .format() returns a PLAIN STRING — no role, no structure
result = pt.format(subject="photosynthesis", grade="5")

print(result)
# Output: Explain photosynthesis suitable for a grade 5 student.

print(type(result))
# Output: <class 'str'>
#
# Key point: PromptTemplate produces a SINGLE STRING.
# The model receives this as raw text with no role context.
```

OUTPUT

```
Explain photosynthesis suitable for a grade 5 student.
<class 'str'>
```

te — Produces a list of messages

Multiple roles (system/human/ai), suited for all modern chat models

EXAMPLE 8b — ChatPromptTemplate output (list of messages)

```
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# EXAMPLE 8b: ChatPromptTemplate — what it produces
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

from langchain.prompts import ChatPromptTemplate

# Define a template with two variables — same variables as above
cpt = ChatPromptTemplate.from_messages([
# Role 1: system message sets the AI's persona
("system", "You are a teacher for grade {grade} students."),

# Role 2: human message asks the question
("human", "Explain {subject} in simple terms.")
])

# .format_messages() returns a LIST OF MESSAGE OBJECTS — not a string
messages = cpt.format_messages(subject="photosynthesis", grade="5")

# Inspect each message object
for msg in messages:
print(f"Type : {type(msg).__name__}")
print(f"Role : {msg.type}")
print(f"Content: {msg.content}")
print("---")

# Output:
# Type : SystemMessage
# Role : system
# Content: You are a teacher for grade 5 students.
# ---
# Type : HumanMessage
# Role : human
# Content: Explain photosynthesis in simple terms.
# ---
#
# Key point: ChatPromptTemplate produces a LIST of typed message objects.
# Each has a ROLE — this is what modern chat APIs expect.
```

OUTPUT

```
Type : SystemMessage
Role : system
Content: You are a teacher for grade 5 students.
---
Type : HumanMessage
```

```
Role : human
Content: Explain photosynthesis in simple terms.
```

Complete comparison table

	PromptTemplate	ChatPromptTemplate
Import	from langchain.prompts import PromptTemplate	from langchain.prompts import ChatPromptTemplate
Output of format()	A plain Python string	A list of message objects (SystemMessage, HumanMessage, etc.)
Message roles	None — single flat text	system, human, ai roles supported
Model type	Completion models (older APIs)	Chat models — GPT-4, Claude, Gemini (modern standard)
How model sees it	Raw text with no context	Structured multi-turn conversation
Variable syntax	{variable} in a string	{variable} inside each message tuple
Method to build	.format(**kwargs)	.format_messages(**kwargs)
Supports system msg?	No	Yes — first-class support
In a chain	prompt | llm	prompt | llm (same pipe syntax)
invoke() input	chain.invoke({'var': value})	chain.invoke({'var': value}) (same API!)
Best used when	Simple, legacy, or non-chat tasks	Almost everything modern — default choice

■ **Rule of thumb:** If you're working with any modern chat model (GPT-4, Claude, Gemini), always use **ChatPromptTemplate**. Use **PromptTemplate** only if you have a specific reason to produce a flat string (e.g. feeding into a non-chat API or a text completion model).

7. Real-World Examples

Here are three complete, practical examples showing ChatPromptTemplate with multiple variables used in real scenarios.

Code reviewer bot (3 variables)

EXAMPLE 9 — Code reviewer bot with 3 variables

```
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# EXAMPLE 9: Code Reviewer Bot
# Variables: {language}, {code}, {focus}
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

from langchain.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

# Template with 3 variables
prompt = ChatPromptTemplate.from_messages([
# System message: define the reviewer's role and expertise
("system",
"You are an expert {language} code reviewer. "
"Be concise, constructive, and specific in your feedback."),

# Human message: provide the code and specify review focus
("human",
"Please review the following {language} code. "
"Focus especially on: {focus}\n\n"
"```{language}\n{code}\n```")
])

llm = ChatOpenAI(model="gpt-4o-mini", api_key="YOUR_API_KEY")
parser = StrOutputParser()

# Build the full chain
chain = prompt | llm | parser

# ■■ Usage ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
review = chain.invoke({
"language": "Python", # programming language
"focus": "performance and readability",# what to emphasise
"code": """
def find_duplicates(lst):
result = []
for i in range(len(lst)):
for j in range(i + 1, len(lst)):
```

```
if lst[i] == lst[j] and lst[i] not in result:
result.append(lst[i])
return result
"""
})
print(review)
```

Email writer (4 variables)

EXAMPLE 10 — Email writer with 4 variables

```
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# EXAMPLE 10: Professional Email Writer
# Variables: {sender_role}, {recipient}, {tone}, {subject}
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
from langchain.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser
# 4-variable template for writing professional emails
prompt = ChatPromptTemplate.from_messages([
# System: establish the writer's persona
("system",
"You are a professional communication assistant helping a {sender_role}. "
"Always write in a {tone} tone. Keep emails concise and actionable."),
# Human: provide the specific email request
("human",
"Write a professional email to {recipient} about: {subject}. "
"Include a clear subject line and sign off appropriately.")
])
chain = prompt | ChatOpenAI(model="gpt-4o-mini", api_key="YOUR_API_KEY") |
StrOutputParser()
# Invoke with all 4 variables
email = chain.invoke({
"sender_role": "project manager", # who is writing
"recipient": "the development team", # who receives it
"tone": "friendly but professional", # writing style
"subject": "sprint review meeting scheduled for Friday 3pm" # email topic
})
print(email)
```

Multi-language translator (3 variables)

EXAMPLE 11 — Multi-language translator with 3 variables

```
# ████████████████████████████████████████████████████████████████████
# EXAMPLE 11: Multi-language Translator with Context
# Variables: {source_lang}, {target_lang}, {text}
# ████████████████████████████████████████████████████████████████████

from langchain.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

# Template: translate text between any two languages
prompt = ChatPromptTemplate.from_messages([
# System: define translation expertise and constraints
("system",
"You are a professional translator fluent in {source_lang} and {target_lang}. "
"Translate accurately while preserving tone, style, and meaning. "
"Return ONLY the translated text — no explanations."),

# Human: provide the text to translate
("human",
"Translate the following from {source_lang} to {target_lang}:\n\n{text}")
])

chain = (
prompt
| ChatOpenAI(model="gpt-4o-mini", api_key="YOUR_API_KEY")
| StrOutputParser()
)

# Translate from English to Spanish
translation = chain.invoke({
"source_lang": "English", # original language
"target_lang": "Spanish", # target language
"text": "The quick brown fox jumps over the lazy dog." # text to translate
})

print(translation)
# Output: El zorro marrón rápido salta sobre el perro perezoso.
```

9. PydanticOutputParser — Typed Python Objects from LLM Output

So far every chain returned either an **AIMessage** or a plain **string**. But what if you need the model to return *structured data* — like a Python object with specific fields you can access by name?

That's exactly what **PydanticOutputParser** does. You define a **Pydantic model** (a typed schema class) and LangChain instructs the LLM to return JSON that matches it. The parser then validates and deserialises the raw JSON string into a real Python object — with type checking, dot-notation access, and IDE autocomplete.

How it works — 3 steps

Step	What you do	LangChain does
1	Define a Pydantic model (your schema)	Nothing yet
2	Create PydanticOutputParser(pydantic_object=YourModel)	Generates format instructions for the LLM
3	chain.invoke({'var': value, ...})	Sends format instructions + your prompt, parses JSON response into Your

Example: Parse a book recommendation into a typed object

EXAMPLE 12 — PydanticOutputParser: book recommendation schema

```
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# EXAMPLE 12: PydanticOutputParser — Basic usage
# Ask the LLM for a book recommendation, get back a typed object
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

from langchain.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain.output_parsers import PydanticOutputParser
from pydantic import BaseModel, Field # Pydantic v2

# ■■ Step 1: Define your schema as a Pydantic model ■■■■■■■■■■■■■■■■■■
# Each field has a type annotation and a Field(description=...).
# The description helps the LLM understand what to put in each field.
class BookRecommendation(BaseModel):
title: str = Field(description="Title of the recommended book")
author: str = Field(description="Full name of the author")
genre: str = Field(description="Genre of the book, e.g. 'science fiction'")
year: int = Field(description="Year the book was published")
summary: str = Field(description="One-sentence summary of the book")
why_read_it: str = Field(description="One sentence explaining why this book suits the
user")
```

```
# ■■ Step 2: Create the parser ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Pass your Pydantic model class — NOT an instance — to the parser.
parser = PydanticOutputParser(pydantic_object=BookRecommendation)

# ■■ Step 3: Build the prompt — inject format instructions ■■■■■■■■■■■
# parser.get_format_instructions() returns a string that tells the
# LLM exactly what JSON structure to produce.
# We inject it using a {format_instructions} variable in the prompt.
prompt = ChatPromptTemplate.from_messages([
("system",
"You are a knowledgeable librarian who recommends books.\n"
"{format_instructions}"), # <-- injects JSON schema instructions
("human",
"Recommend a {genre} book suitable for a {reader_level} reader.")
])

# ■■ Step 4: Create the model ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0, api_key="YOUR_API_KEY")
# temperature=0 → deterministic output, important for structured parsing

# ■■ Step 5: Build the full chain ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# prompt → llm → parser
# The parser automatically validates and converts JSON → BookRecommendation
chain = prompt | llm | parser

# ■■ Step 6: Invoke the chain ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Note: format_instructions is passed as a variable alongside your own vars
result = chain.invoke({
"format_instructions": parser.get_format_instructions(), # schema hint
"genre": "classic science fiction", # your variable
"reader_level": "adult" # your variable
})

# ■■ Step 7: Use the typed result ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# result is a BookRecommendation object — not a string!
print(type(result)) # <class '__main__.BookRecommendation'>

# Access fields by name with dot notation
print(result.title) # e.g. "Dune"
print(result.author) # e.g. "Frank Herbert"
print(result.genre) # e.g. "science fiction"
print(result.year) # e.g. 1965 (an int, not a string!)
print(result.summary) # e.g. "A saga set on a desert planet..."
print(result.why_read_it) # e.g. "A perfect entry into epic sci-fi..."

# Convert to dict if needed
print(result.model_dump()) # {'title': 'Dune', 'author': 'Frank Herbert', ...}
```

```
# Convert to JSON string if needed
print(result.model_dump_json()) # '{"title": "Dune", "author": ...}'
```

result is a typed Python object — access fields with dot notation

OUTPUT

```
<class '__main__.BookRecommendation'>
title : Dune
author : Frank Herbert
genre : science fiction
year : 1965
summary : A saga set on the desert planet Arrakis, following Paul Atreides...
why_read_it: A foundational epic that defined the science fiction genre.
```

Example: Richer schema with Optional fields and lists

EXAMPLE 13 — Richer schema with List and Optional fields

```
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# EXAMPLE 13: More complex schema — Optional fields and a list
# Parse a product review into structured fields
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

from langchain.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain.output_parsers import PydanticOutputParser
from pydantic import BaseModel, Field
from typing import List, Optional

# ■■ Schema: structured product review ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
class ProductReview(BaseModel):
product_name: str = Field(description="Name of the product being reviewed")
rating: int = Field(description="Rating from 1 to 5", ge=1, le=5)
pros: List[str] = Field(description="List of positive aspects, max 3 items")
cons: List[str] = Field(description="List of negative aspects, max 3 items")
verdict: str = Field(description="One-word verdict: 'Recommended', "
"'Neutral', or 'Avoid'")
best_for: Optional[str] = Field(
default=None,
description="Who this product is best suited for, or null")

# ■■ Parser and prompt ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
parser = PydanticOutputParser(pydantic_object=ProductReview)

prompt = ChatPromptTemplate.from_messages([
```

```
("system",
"You are a professional product reviewer. Be balanced and fair.\n"
"{format_instructions}"),
("human",
"Write a structured review of the {product} based on its {use_case} use case.")
])

llm = ChatOpenAI(model="gpt-4o-mini", temperature=0, api_key="YOUR_API_KEY")
chain = prompt | llm | parser

# ■■ Invoke ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
review = chain.invoke({
"format_instructions": parser.get_format_instructions(),
"product": "Sony WH-1000XM5 headphones",
"use_case": "remote work and focus"
})

# ■■ Access typed fields ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
print(f"Product : {review.product_name}")
print(f"Rating : {review.rating}/5") # guaranteed int
print(f"Pros : {review.pros}") # guaranteed list
print(f"Cons : {review.cons}") # guaranteed list
print(f"Verdict : {review.verdict}")
print(f"Best for: {review.best_for}") # Optional — may be None
```

OUTPUT

```
Product : Sony WH-1000XM5 headphones
Rating : 5
Pros : ['Exceptional noise cancellation', 'Comfortable for long sessions', 'Clear call quality']
Cons : ['Premium price', 'Carries case is bulky']
Verdict : Recommended
Best for: Remote workers and students who need deep focus
```

Example: PydanticOutputParser with 3 prompt variables

EXAMPLE 14 — Full chain: 3 variables + PydanticOutputParser

```
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# EXAMPLE 14: Full chain — ChatPromptTemplate (3 variables)
# + PydanticOutputParser → typed Python object
# Parse a programming concept explanation into structured data
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

from langchain.prompts import ChatPromptTemplate
```

```
from langchain_openai import ChatOpenAI
from langchain.output_parsers import PydanticOutputParser
from pydantic import BaseModel, Field
from typing import List

# ■■ Step 1: Define the output schema ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
class ConceptExplanation(BaseModel):
concept: str = Field(description="The programming concept name")
definition: str = Field(description="Clear one-sentence definition")
analogy: str = Field(description="A real-world analogy to explain it simply")
code_example: str = Field(description="A short Python code snippet illustrating the concept")
key_points: List[str] = Field(description="3 key points to remember, as a list")
difficulty: str = Field(description="Difficulty level: 'Beginner', 'Intermediate', or 'Advanced'")

# ■■ Step 2: Create the parser ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
parser = PydanticOutputParser(pydantic_object=ConceptExplanation)

# ■■ Step 3: Build the prompt with 3 custom variables + format_instructions
# Variables:
# {concept} — the topic to explain
# {language} — the programming language context
# {audience} — who the explanation is aimed at
# {format_instructions} — injected by parser (always required)
prompt = ChatPromptTemplate.from_messages([
("system",
"You are an expert {language} instructor who specialises in teaching {audience}.\n"
"Always use clear, practical examples.\n\n"
"{format_instructions}"),
("human",
"Explain the {language} concept of '{concept}' to a {audience}.")
])

# ■■ Step 4: Build and run the chain ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0, api_key="YOUR_API_KEY")
chain = prompt | llm | parser

result = chain.invoke({
# Parser-required variable — always include this
"format_instructions": parser.get_format_instructions(),

# Your own custom variables
"concept": "closures", # what to explain
"language": "Python", # language context
"audience": "junior developer" # target reader
})
```

```
# ■■ Step 5: Use the typed result ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
print(f"Concept : {result.concept}")
print(f"Definition: {result.definition}")
print(f"Analogy : {result.analogy}")
print(f"Difficulty: {result.difficulty}")
print(f"\nCode Example:\n{result.code_example}")
print(f"\nKey Points:")
for i, point in enumerate(result.key_points, 1): # result.key_points is a List[str]
print(f" {i}. {point}")
```

format_instructions is always a required variable when using PydanticOutputParser

StrOutputParser vs PydanticOutputParser — when to use which

	StrOutputParser	PydanticOutputParser
Returns	A plain Python str	A typed Pydantic model instance
Access data	String — parse manually if needed	result.field_name dot notation
Type safety	None	Full — int stays int, list stays list
Schema required	No	Yes — define a BaseModel class
format_instructions	Not needed	Must pass parser.get_format_instructions()
Best for	Prose answers, summaries, explanations	Structured data, APIs, data pipelines
Failure mode	Rarely fails	Can fail if LLM output is malformed JSON

■ **Always pass format_instructions as a variable.** When using PydanticOutputParser, call **parser.get_format_instructions()** and include it in your invoke() dict under the key matching the **{format_instructions}** placeholder in your prompt. Without it, the LLM won't know what JSON structure to produce.

■ **Tip — set temperature=0 with PydanticOutputParser.** Structured output parsing works best when the model is deterministic. A higher temperature can cause the LLM to produce malformed JSON that fails to parse. Use temperature=0 for reliable results.

8. Key Rules & Common Mistakes

Rules to always follow

1. Variable names must match exactly

The key in chain.invoke({'var': value}) must match the {var} name in the template. A typo causes a KeyError at runtime.

2. Every variable must be provided

If your template has {topic}, {level}, and {language}, all three must be in the invoke dict. Missing any one raises a KeyError.

3. Variables work across all messages

A variable like {language} defined in invoke() fills {language} in every message — system, human, and AI — automatically.

4. ChatPromptTemplate is the default choice

For all modern chat models use ChatPromptTemplate. PromptTemplate is for legacy/completion-style models only.

5. budget_tokens does NOT apply here

LangChain prompts have nothing to do with Claude's budget_tokens. That is a separate Claude API concept.

Common mistakes and how to fix them

Mistake	What goes wrong	Fix
Using PromptTemplate with a chat model	Model still works but you lose the system message — answers are inconsistent	Switch to ChatPromptTemplate
Variable name mismatch e.g. {Topic} vs 'topic'	KeyError: 'Topic' — Python keys are case-sensitive	Keep variable names lowercase, match exactly
Missing a variable in invoke() e.g. forgetting 'level'	KeyError: 'level' — LangChain cannot fill the placeholder	Include every {variable} as a key in the dict
Forgetting StrOutputParser	chain.invoke() returns AIMessage — you must call .content to get the text	Add | StrOutputParser() to the chain
Setting temperature with extended thinking (Claude only)	API error — incompatible parameters	Remove temperature when using Claude extended thinking

Summary

Concept	Key Point
PromptTemplate	Produces a plain string. Use for simple or non-chat models.
ChatPromptTemplate	Produces a list of role-based messages. Default for modern chat models.
Defining variables	Write {var_name} inside the message strings.
Format method	PromptTemplate uses .format(). ChatPromptTemplate uses .format_messages().
Building a chain	prompt | llm or prompt | llm | StrOutputParser()
Invoking the chain	chain.invoke({'var1': value, 'var2': value, ...})
Variable matching	Keys in invoke() dict must match {variable} names exactly.
All vars required	Every {variable} in the template must appear in invoke().
StrOutputParser	Append | StrOutputParser() to get a plain string from the chain.
Roles available	"system", "human", "ai" — use system for the AI persona.
Multiple variables	No limit — add as many {variables} as needed across messages.
PydanticOutputParser	Define a BaseModel schema; parser converts JSON → typed Python object.
format_instructions var	Always pass parser.get_format_instructions() in invoke() dict.
Pydantic temperature tip	Use temperature=0 for reliable structured JSON output.

■ You now know the fundamentals. Start with **Example 4** (2 variables, core pattern), then try **Example 7** (StrOutputParser). Use **Example 9–11** as real-world templates. For structured data, study **Example 12–14** (PydanticOutputParser).